

Universidade Cidade de São Paulo - UNICID
Curso de Ciência da Computação

ERIC CONDE DERRE
ERICK DO CARMO ESTEVES
FERNANDO MATOS DE SOUZA
GUILHERME PENHA DOS SANTOS
JOÃO VICTOR FONSECA SILVA
KENNEDY FERNANDO DE OLIVEIRA GUNDIM
LEANDRO HENRIQUE DA SILVA PATRICIO
PEDRO HENRIQUE TORRES GONÇALVES
RAUL SILVESTRE MARINHO

DESENVOLVIMENTO DE UM AGENTE DE QA END-TO-END (E2E) AUTÔNOMO
COM CAPACIDADES DE SELF-HEALING:
Focado na Fronteira de Integração

São Paulo
2026

ERIC CONDE DERRE
ERICK DO CARMO ESTEVES
FERNANDO MATOS DE SOUZA
GUILHERME PENHA DOS SANTOS
JOÃO VICTOR FONSECA SILVA
KENNEDY FERNANDO DE OLIVEIRA GUNDIM
LEANDRO HENRIQUE DA SILVA PATRICIO
PEDRO HENRIQUE TORRES GONÇALVES
RAUL SILVESTRE MARINHO

**DESENVOLVIMENTO DE UM AGENTE DE QA END-TO-END (E2E) AUTÔNOMO
COM CAPACIDADES DE SELF-HEALING:**

Focado na Fronteira de Integração

Projeto de pesquisa apresentado ao curso de
Ciência da Computação da Universidade Cidade de
São Paulo (UNICID), como requisito parcial para a
elaboração do Trabalho de Conclusão de Curso.

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO

- 1.1 Contextualização e Delimitação do Tema
- 1.2 Problema de Pesquisa
- 1.3 Justificativa
- 1.4 Objetivos

2 REFERENCIAL TEÓRICO

- 2.1 Evolução da Automação E2E e o Conceito de Self-Healing
- 2.2 Fundamentos de Aprendizado de Máquina e Adaptação
- 2.3 Automação E2E e Inteligência Artificial
- 2.4 Gestão de Context Window em LLMs

3 METODOLOGIA

- 3.1 Infraestrutura e Orquestração Local
- 3.2 Inteligência Artificial e Processamento LLM
- 3.3 Comunicação e Mitigação de Gargalos
- 3.4 Validação e Avaliação

4 RESULTADOS ESPERADOS

5 CRONOGRAMA DE ATIVIDADES

REFERÊNCIAS

GLOSSÁRIO

1 INTRODUÇÃO

1.1 Contextualização e Delimitação do Tema

A automação de testes End-to-End (E2E) tradicional enfrenta um alto custo de manutenção devido à fragilidade crônica de seus scripts. Uma alteração simples no CSS do Frontend ou a mudança invisível do tipo de um dado retornado por uma API, fenômeno conhecido como *Contract Drift* (Deriva de Contrato), quebra instantaneamente a esteira de CI/CD (Integração Contínua/Entrega Contínua), gerando perda de tempo em triagens manuais exaustivas.

Embora a automação moderna baseada em Inteligência Artificial ofereça avanços, a literatura e a prática industrial demonstram que a IA não tem o intuito de substituir os engenheiros de qualidade (QA) ou os desenvolvedores, mas sim assumir as tarefas repetitivas. Dessa forma, os profissionais mantêm o controle sobre decisões de risco, estratégias de cobertura e testes exploratórios complexos.

Neste escopo, o presente projeto delimita-se ao desenvolvimento de um Agente de QA End-to-End (E2E) autônomo com capacidades de *Self-Healing* (autocura), focado estritamente na fronteira de integração (Front-to-Back) da API.

1.2 Problema de Pesquisa

Diante do cenário de fragilidade dos testes automatizados e das limitações de infraestrutura corporativa, levanta-se a seguinte questão de pesquisa: Como um Modelo de Linguagem Grande (LLM), aliado à interceptação bidimensional, pode diagnosticar autonomamente falhas na fronteira de integração Front-to-Back, sem a necessidade de acoplamento à lógica interna do servidor, operando sob restrições de hardware local?

1.3 Justificativa

Este trabalho justifica-se pela necessidade de desenvolver ferramentas de assistência focadas em acelerar a Análise de Causa Raiz para as equipes técnicas. Tradicionalmente, quando ocorre uma falha silenciosa na integração cliente-servidor, perdem-se horas cruzando capturas de tela com logs de servidor para entender o

desvio.

O agente proposto atuará como um copiloto de depuração inteligente: ao unir a renderização gráfica à interceptação nativa de rede do framework Playwright, ele pré-processa a falha e entrega ao desenvolvedor um diagnóstico rápido de onde o contrato da API divergiu do esperado pela interface. O projeto visa melhorar o tempo de resposta da equipe frente a bugs de API e reduzir o desgaste com a manutenção de rotinas frágeis, permitindo que o profissional foque na revisão da qualidade e na garantia da integridade da arquitetura de software.

1.4 Objetivos

Objetivo Geral: Desenvolver, implementar e mensurar a eficácia de um Agente de QA E2E Autônomo, utilizando LLMs em arquitetura de nuvem híbrida para realizar o *Self-Healing* de scripts, propor correções de código baseadas na análise conjunta de interfaces e interceptação de rede, e facilitar o seu debug.

Objetivos Específicos:

- Levantar a fundamentação teórica e preparar o ambiente base, pesquisando metodologias de automação com LLMs, extração de árvores de acessibilidade e quebra de contrato de APIs.
- Prototipar a topologia de nuvem híbrida e comunicação, configurando orquestrador local e infraestrutura em nuvem (Google Colab com FastAPI e Ngrok) para alocar a inferência do modelo Qwen2.5-Coder.
- Implementar o Módulo de Filtragem de Contexto (DOM Pruning) para impedir o estouro da janela de contexto (Context Window) do LLM.
- Desenvolver o Agente de Interceptação e Self-Healing, integrando a automação visual para capturar requisições HTTP e codificar a lógica autônoma de correção.
- Mensurar e validar a precisão e performance do agente contra aplicações de teste, utilizando injeção de falhas sintéticas e Fuzzing de mutação de API.

2 REFERENCIAL TEÓRICO

Este projeto fundamenta-se nos avanços documentados na literatura técnico-científica

sobre a adoção de modelos generativos no ciclo de vida de desenvolvimento de software (SDLC) e nos fundamentos clássicos de aprendizado de máquina.

2.1 Evolução da Automação E2E e o Conceito de Self-Healing

A automação de testes de software transitou de scripts manuais lineares para abordagens complexas de ponta a ponta (End-to-End - E2E), projetadas para simular o comportamento de um usuário real em toda a pilha tecnológica da aplicação. Inicialmente, a automação E2E baseava-se em localizadores estáticos dentro do Document Object Model (DOM) — a estrutura HTML da página. Contudo, essa dependência tornava os testes frágeis e suscetíveis a quebras mediante qualquer alteração visual ou estrutural na interface gráfica.

Para mitigar este problema, o conceito de autocura (*Self-Healing*) emergiu na interseção entre engenharia de software e inteligência artificial. Em sistemas distribuídos, a autocura refere-se à capacidade de um sistema detectar falhas ativamente, diagnosticar sua causa raiz e aplicar correções de forma autônoma. No contexto de testes de qualidade (QA), um agente dotado de *Self-Healing* utiliza algoritmos de busca e processamento de linguagem natural para identificar dinamicamente novos localizadores na interface ou interpretar desvios nas respostas do servidor (*Contract Drift*) quando o script original falha, restaurando assim o fluxo de execução sem a necessidade de intervenção humana imediata.

2.2 Fundamentos de Aprendizado de Máquina e Adaptação

A transição de sistemas baseados em regras estritas para sistemas adaptativos (como os testes com *Self-Healing*) encontra sua base teórica no aprendizado de máquina. Segundo Faceli et al. (2021), os algoritmos de aprendizado de máquina permitem que sistemas computacionais extraiam padrões intrínsecos a partir de dados para a tomada de decisão autônoma, superando as limitações da programação puramente determinística. Ao invés de falhar imediatamente ante uma mudança no DOM, o sistema dotado de inteligência artificial utiliza o reconhecimento de padrões para inferir

a intenção original do teste e sugerir a correção.

2.3 Automação E2E e Inteligência Artificial

Estudos recentes, como o GenIA-E2ETest (JÚNIOR et al., 2025), compõem a base metodológica que atesta a eficácia dos LLMs (Modelos de Linguagem Grandes) na interpretação de intenções textuais e tradução para comandos automatizados de interação de elementos de interface. A implementação do Playwright Healer Agent (BISWAS, 2026) sustenta o diferencial arquitetural deste trabalho, validando que a interceptação do tráfego de rede aliada à IA mitiga drasticamente os falsos-positivos.

2.4 Gestão de Context Window em LLMs

A literatura comprova que sobrecarregar a memória de contexto (Context Window) dos LLMs com códigos HTML puros degrada seu raciocínio lógico, um fenômeno conhecido como Attention Dilution. Metodologias focadas em podar e limpar a árvore de elementos antes do processamento, como o Prune4Web (ZHANG et al., 2026), formam a espinha dorsal das técnicas de filtragem utilizadas neste projeto para viabilizar inferências precisas com custo computacional reduzido.

3 METODOLOGIA

A pesquisa caracteriza-se como um projeto prático e aplicado (Pesquisa-Ação), voltado para a construção e validação de uma arquitetura de software escalável. A infraestrutura baseia-se nas seguintes camadas:

3.1 Infraestrutura e Orquestração Local

A ferramenta principal de orquestração será o Playwright (com Node.js/Python). O Playwright comunica-se com o navegador via protocolos nativos e possui suporte embutido para interceptação e modificação de chamadas de rede em tempo real. Isso permite extrair a árvore de acessibilidade do DOM e escutar os códigos de status HTTP em segundo plano. Como contingência, o framework Cypress poderá ser acionado caso ocorram incompatibilidades críticas.

3.2 Inteligência Artificial e Processamento LLM

O processamento semântico utilizará o modelo Qwen2.5-Coder, executado no ambiente de nuvem do Google Colab (utilizando GPU T4 com 16GB VRAM), contornando as restrições de hardware locais e garantindo alta capacidade de raciocínio lógico focado em programação e depuração de código.

3.3 Comunicação e Mitigação de Gargalos

A integração ocorrerá via uma interface de programação de aplicações construída com o framework FastAPI, acompanhada de um túnel seguro de rede (Ngrok). Para evitar o colapso da janela de contexto do LLM, aplicar-se-ão heurísticas estritas de DOM Pruning, enviando apenas os dados limpos e essenciais da interface e realizando o truncamento de dados JSON muito extensos interceptados na rede.

3.4 Validação e Avaliação

A avaliação evitará a subjetividade inerente às análises feitas puramente por IAs, adotando aplicações-alvo determinísticas. Aplicar-se-á técnicas de Fuzzing (injeção de dados anômalos e imprevistos) e Mutação Dinâmica para simular quebras de contrato reais em APIs. O sucesso do agente será avaliado quantitativamente por duas métricas principais: Tempo Médio de Reparo (MTTR) e Task Success (Sucesso da Tarefa), verificando na prática se a sugestão refatorada consegue restaurar a estabilidade do teste.

4 RESULTADOS ESPERADOS

Espera-se que o Agente de QA Autônomo reduza significativamente o Tempo Médio de Reparo (MTTR) associado à manutenção de testes E2E. Projetamos que o agente identifique com precisão a causa raiz de falhas de comunicação através da depuração bidimensional. Adicionalmente, o projeto deve comprovar a viabilidade técnica da orquestração de IA avançada em máquinas com restrições de hardware através da topologia de nuvem híbrida proposta.

5 CRONOGRAMA DE ATIVIDADES

Etapa / Mês	Atividade Prevista
Mês 1	Revisão bibliográfica (LLMs, DOM Pruning, Contract Drift).
Mês 2	Configuração da infraestrutura local e nuvem (Google Colab, FastAPI, Ngrok).
Mês 3	Desenvolvimento do Módulo de Filtragem de Contexto e extração da Accessibility Tree via Playwright.
Mês 4	Codificação do fluxo de Self-Healing e lógica de interceptação do Agente.
Mês 5	Testes práticos (Fuzzing) em aplicações-alvo e coleta de métricas (MTTR e Task Success).
Mês 6	Redação, formatação ABNT e revisão dos capítulos do TCC.
Mês 7	Finalização do documento e Defesa Final perante a banca.

REFERÊNCIAS

BISWAS, S. Enhancing End-to-End Test Stability Through AI-Assisted Self-Healing: A Case Study of Playwright Healer Agent Implementation. **International Journal of Scientific Engineering and Research**, v. 14, n. 1, p. 1-12, jan. 2026.

FACELI, K.; LORENA, A. C.; GAMA, J.; ALMEIDA, T. A.; CARVALHO, A. C. P. L. F. **Inteligência Artificial: uma abordagem de aprendizado de máquina**. 2. ed. Rio de Janeiro: LTC, 2021.

JOSEPH, R. N. Beyond LLM-Based Test Automation: A Zero-Cost Self-Healing Approach Using DOM Accessibility Tree Extraction. **Preprint**, mar. 2026.

JÚNIOR, E.; VALEJO, A. D. B.; VALVERDE-REBAZA, J.; NEVES, V. GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation. In: **Simpósio Brasileiro de Engenharia de Software (SBES)**, Recife, PE, set. 2025.

MACIAK, T. et al. **Automated contract testing: How to detect API drift before it reaches production**. Medium, 2026.

SHI, X.; LI, Z.; CHEN, A. R. Enhancing LLM-based Fault Localization with a Functionality-Aware Retrieval-Augmented Generation Framework. **arXiv preprint**, 2025.

ZHANG, J. et al. Prune4Web: DOM Tree Pruning Programming for Web Agent. In: **AAAI Conference on Artificial Intelligence**, 2026.

GLOSSÁRIO

API (Application Programming Interface): Conjunto de definições e protocolos padronizados que permite a comunicação e a troca de dados entre diferentes sistemas de software.

Contract Drift (Deriva de Contrato): Alteração silenciosa ou não documentada na estrutura dos dados retornados por uma API (como a mudança de um tipo numérico para texto), quebrando as expectativas do cliente (interface) que consome esses dados.

DOM (Document Object Model): Representação estruturada em forma de árvore do conteúdo de um documento HTML, utilizada pelos navegadores web para renderizar páginas e permitir interações via scripts.

End-to-End (E2E): Abordagem de teste de software que valida o fluxo completo de uma aplicação, de ponta a ponta, simulando o comportamento de um usuário real desde a interface gráfica até a camada de banco de dados.

FastAPI: Framework web moderno e de alto desempenho, utilizado na linguagem Python para a construção e orquestração ágil de APIs.

Fuzzing: Técnica de teste de software que envolve fornecer dados inválidos, inesperados ou aleatórios como entrada para um sistema, a fim de expor vulnerabilidades, falhas de validação ou exceções não tratadas.

LLM (Large Language Model - Modelo de Linguagem Grande): Modelo avançado de inteligência artificial treinado em vastas quantidades de texto, capaz de compreender o contexto, raciocinar e gerar linguagem natural e código de programação estruturado.

Ngrok: Ferramenta de conectividade de rede multiplataforma que cria um túnel seguro, expondo temporariamente um servidor executado em máquina local para a internet pública.

Playwright / Cypress: Frameworks modernos de automação de testes para aplicações

web. Ambos permitem a simulação robusta de interações de usuários e possuem capacidades nativas de interceptação de tráfego de rede no navegador.

QA (Quality Assurance - Garantia de Qualidade): Conjunto de atividades preventivas e processos de engenharia de software focados em garantir que a aplicação atenda aos requisitos especificados e aos padrões operacionais antes do lançamento em produção.

Qwen2.5-Coder: Variante de modelo de linguagem de código aberto otimizada especificamente para tarefas de programação, tradução de sintaxe, depuração e raciocínio lógico voltado para software.

Self-Healing (Autocura): Capacidade intrínseca de um sistema distribuído, ou de um script de automação, de detectar ativamente falhas de execução em tempo real e aplicar correções estruturais autonomamente para continuar operando sem interrupções.